REQUIREMENTS

Mei Nagappan



UNIVERSITY OF WATERLOO | FACULTY OF MATHEMATICS

Guideline 4: Provide a Structured Output Template

- When asking the model to draft an SRS section, include the exact section template (headings, field names, and accepted values) in the prompt. Do not ask for free-form prose. If you do not supply a template, the model will invent one from its training data — which will not match your tracker or tooling.

Reasoning

- Predefined templates improve LLM-generated FDS quality significantly; free-form generation required heavy revision.
- Template-constrained subtasks for SRS generation reduce hallucination and improve non-redundancy scores.
- Output Template Pattern: explicit output structure prevents field invention and generic documentation boilerplate.

Key Practices: Prompting with a Template

- Paste your organization's actual SRS template verbatim into the prompt before any requirement content. Do not ask for 'an IEEE 830 template' — there is no single standard layout and the model will improvise one.
- Specify field names and accepted values explicitly: e.g., "Priority: [High | Medium | Low]", "Verification: [Test | Inspection | Analysis | Demonstration]".
- Ask the model to fill in one requirement per template block; do not ask for a batch of requirements in one unstructured block.

Key Practices: Reviewing Output

- Confirm that no field was left blank or filled with a placeholder such as "TBD" without flagging it for follow-up.
- Verify that the "Verification" field for each requirement is a method your team can realistically execute.
- "Priority" must be justified by business rationale, not left as a default.

Guideline 5: Detect Conflicts and Undefined Terms

- Once a draft requirements set is complete, feed the entire list to the model in one prompt and ask it to identify contradictions between requirements and flag any term used in more than one requirement without a shared definition.

Reasoning

- Validation is the RE phase with the most consistent LLM contribution because 'find contradictions' is a narrowly defined analytical task rather than generative.

- Structured analytical prompts where the model is given a checklist of what to look for produce the most reliable outputs in RE validation tasks.

Key Practices: Conflict Detection

- Paste the complete numbered requirements list and ask: "Identify any pair of requirements that contradict each other. Then list every domain term that appears in more than one requirement and does not have a shared definition. Output each finding as a table row."
- Use two separate output tables: one for contradictions and one for undefined shared terms.
- Include both functional and non-functional requirements in the same prompt.

Key Practices: Acting on Results

- For each conflict, hold a stakeholder decision meeting before writing any code; do not ask the model to resolve the conflict on its own.
- For each undefined shared term, add a Definitions section to the SRS and run the conflict detection again.
- Treat any conflict between a functional requirement and an NFR as highest priority as these are most likely to cause architectural rewrites.

Guideline 6: Supply a Worked Example for NFRs

- When asking the model to generate non-functional requirements (NFRs), include one concrete, measurable NFR as a worked example directly in the prompt. This anchors output format and prevents untestable quality attributes.

Reasoning

- NFRs score highest when the model is given a template with an example; without one, models fall back to vague adjectives like 'fast' or 'secure'.
- LLM requirements meet quality criteria less frequently than human-authored ones because of language clarity failures.
- Few-shot prompting transfers format and measurability expectations to the model and no fine-tuning is needed.

Key Practices: Prompting with a Worked Example

- Format the example NFR as: "[Quality attribute] -- [Measurable condition] -- [Verification method]"
- Example to include: "Performance -- The search endpoint shall return results within 300 ms at p95 under 200 concurrent requests -- verified by a k6 load test."
- After providing the example, list the FRs you want NFRs derived from and ask the model to follow the same format for each.

Key Practices: Reviewing NFRs

- Delete any NFR that does not contain a numeric threshold or a named standard (e.g., WCAG 2.1 AA, TLS 1.3).
- For each NFR, confirm the verification method is something your team can actually execute.
- Re-check that security and availability NFRs are present for any FR involving user authentication or external data.

References (1 of 2)

- [1] White et al. (2023). ChatGPT prompt patterns for improving code quality, refactoring, requirements elicitation, and software design. arXiv:2303.07839.
- [2] Ronanki et al. (2023). Investigating ChatGPT's potential to assist in requirements elicitation processes. Euromicro SEAA, pp. 354-361. IEEE.
- [3] Ronanki et al. (2024). Requirements engineering using generative AI: Prompts and prompting patterns. In Generative AI for Effective Software Development. Springer. arXiv:2311.03832.
- [4] Quattrocchi et al. (2025). Can LLMs generate user stories and assess their quality? arXiv:2507.15157.
- [5] Almonte et al. (2025). Automated non-functional requirements generation in software engineering with large language models. arXiv:2503.15248.
- [6] Pasquale et al. (2025). Exploring the use of LLMs for requirements specification in an IT consulting company. arXiv:2507.19113.

References (2 of 2)

- [7] Zadenoori et al. (2024). Large language models for requirements engineering: A systematic literature review. arXiv:2509.11446.
- [8] Hemmat et al. (2025). Research directions for using LLM in software requirement engineering: A systematic review. Frontiers in Computer Science. DOI: 10.3389/fcomp.2025.1519437
- [9] Zhu et al. (2025). ReqInOne: A large language model-based agent for software requirements specification generation. arXiv:2508.09648.
- [10] ISO/IEC/IEEE 29148:2018. Systems and Software Engineering -- Life Cycle Processes -- Requirements Engineering. DOI: 10.1109/IEEESTD.2018.8559686
- [11] Khlood et al. (2023). Requirements elicitation and modelling of AI systems. ENASE, pp. 15-26. DOI: 10.5220/0011863200003464
- [12] Habibullah et al. (2023). Non-functional requirements for machine learning. Requirements Engineering, 28(2), 283-316. DOI: 10.1007/s00766-022-00379-z